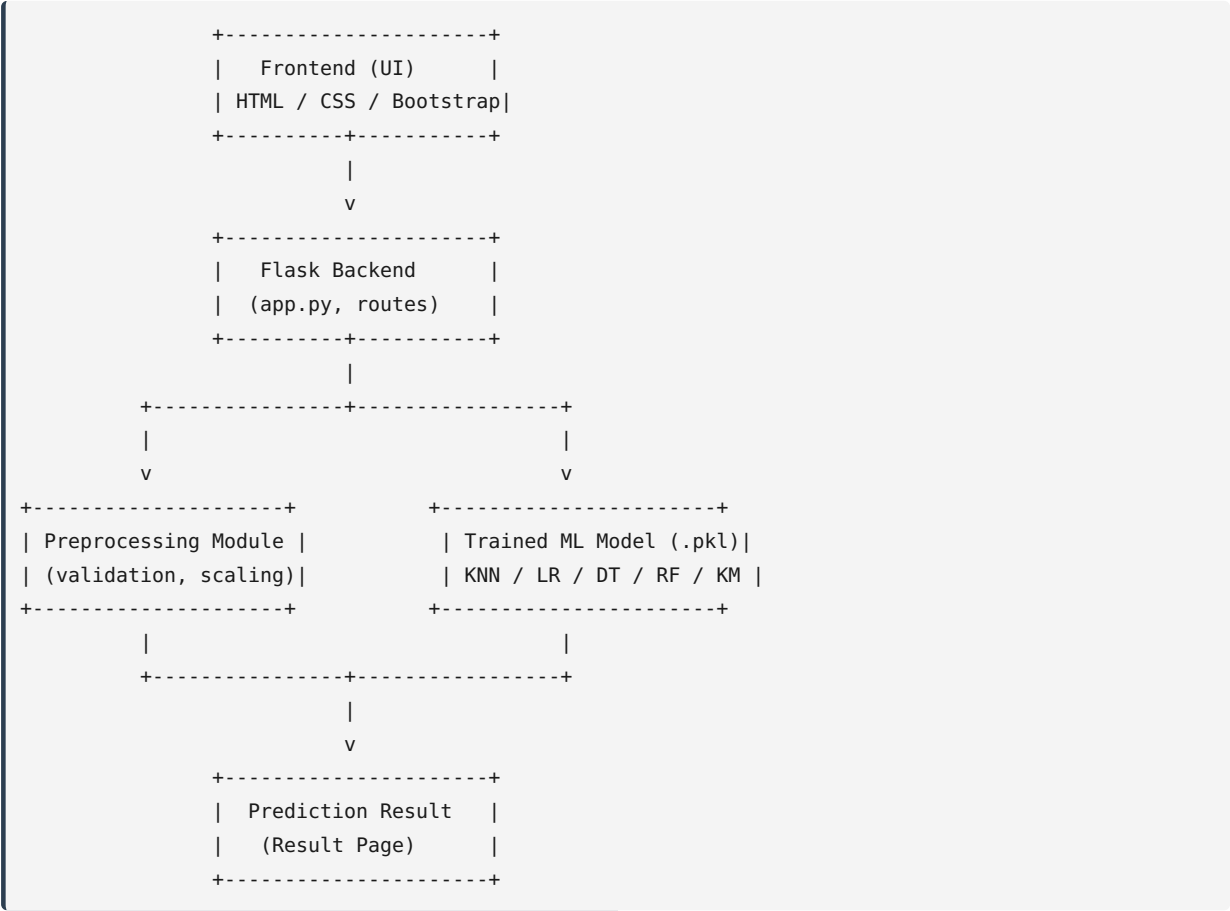


System Design

System Design — OptiCrop

1. System Architecture (High-Level)



2. Module Breakdown

Module	Responsibility
<code>data_preprocessing.py</code>	Cleaning, handling missing values, feature scaling, encoding
<code>model_training.py</code>	Trains and compares KNN, Logistic Regression, Decision Tree, Random Forest, K-Means; selects best model
<code>model_persistence.py</code>	Saves/loads trained model via Pickle (.pkl)
<code>app.py</code> (Flask)	Handles routes, input validation, calls prediction, renders templates
<code>templates/</code>	HTML pages (input form, result page)
<code>static/</code>	CSS/Bootstrap assets

3. Data Flow Diagram (Level 0)

```
[User] --(soil & climate data)--> [Flask App] --(validated input)--> [ML Model]
[ML Model] --(predicted crop)--> [Flask App] --(rendered result)--> [User]
```

4. Data Flow Diagram (Level 1)

1. User submits form with N, P, K, temperature, humidity, pH, rainfall
2. Flask route receives POST request
3. Input validation module checks data types/ranges
4. Preprocessing module scales/transforms input to match model's training format
5. Model (loaded from .pkl) generates prediction
6. Result rendered on results.html page

5. Entity Overview (Dataset Schema)

Field	Type	Description
N	float	Nitrogen content in soil
P	float	Phosphorous content in soil
K	float	Potassium content in soil
temperature	float	Ambient temperature (°C)
humidity	float	Relative humidity (%)
ph	float	Soil pH level
rainfall	float	Rainfall (mm)
label	string	Target crop (used for training only)

6. UI Design Overview

- **Home/Input Page:** Form with 7 numeric input fields + “Predict” button
- **Result Page:** Displays recommended crop name, with option to go back and re-enter values

7. Technology Stack

- **Backend:** Python, Flask
- **ML:** Scikit-learn, NumPy, Pandas, SciPy
- **Visualization (analysis phase):** Matplotlib, Seaborn
- **Frontend:** HTML, CSS, Bootstrap
- **Model Storage:** Pickle (.pkl)
- **Version Control:** Git